# LEXAURA

Submitted in partial fulfillment of the requirements of the degree

**BACHELOR OF ENGINEERING IN ARTIFICIAL INTELLIGENCE & DATA SCIENCE ENGINEERING**


By

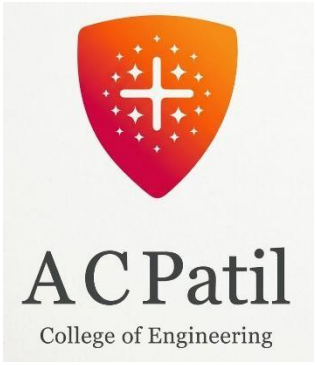
Rushil Pawar (51)

Rahul Sharma (59)

Harshvardhan Wadekar (71)


Guide Name

Prof. A. P. Haral





**Department of Artificial Intelligence & Data Science Engineering**

**A. C. Patil College of Engineering Kharghar, Navi Mumbai**

**University of Mumbai**


**(AY 2026-27)**

# CERTIFICATE

This is to certify that the Mini Project entitled "LexAura: Automated Legal Document & Clause Analyzer" is a bonafide work of **Rushil Pawar (51), Rahul Sharma (59) & Harshvardhan Wadekar (71)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of "**Bachelor of Engineering**" in "**Artificial Intelligence & Data Science Engineering**".

Prof. A. P. Haral

Guide

Prof. S. P. Bansu

Head of Department

Dr. S. R. Deore

Principal

# MINI PROJECT APPROVAL

This Mini Project entitled **"LexAura: Automated Legal Document & Clause Analyzer"** by **Rushil Pawar (51), Rahul Sharma (59) & Harshvardhan Wadekar (71)** is approved for the degree of **Bachelor of Engineering in Artificial Intelligence & Data Science Engineering.**

**Examiners**

1……

(Internal Examiner Name & Sign)

2……

(External Examiner name & Sign)

Date:

Place:

# 1. INTRODUCTION

Commercial agreements such as Master Services Agreements (MSAs), Non-Disclosure Agreements (NDAs), Data Processing Agreements (DPAs), Statements of Work (SOWs), and enterprise software contracts often contain dense legal language, cross-references, obligations, deadlines, and clauses with different levels of business risk. Reviewing these documents manually can be time-consuming and important terms may be overlooked, especially when contracts are lengthy or use inconsistent formatting.

**LexAura: Automated Legal Document & Clause Analyzer** is a Natural Language Processing (NLP) platform intended to help legal engineers, contract review teams, and researchers inspect commercial agreements. It accepts PDF and DOCX files, parses their structure, segments clauses, identifies legal entities and obligations, classifies clauses by risk, and produces a concise summary with references to the source clauses.

The system combines deterministic rule-based processing with transformer-based models where available. Its pipeline includes document validation, clause segmentation, legal named entity recognition (Legal-NER), normalization of dates and notice periods, obligation extraction, multi-tier risk classification, and extractive or abstractive summarization. The backend is implemented using Python and FastAPI, while the web client uses React 19 with Vite. Docker Compose is provided for local orchestration.

LexAura is designed as a decision-support and productivity tool. Its generated results are automated interpretations and must be reviewed by a qualified legal professional; they do not constitute legal advice.

# 2. OBJECTIVES

The main objective of LexAura is to automate selected stages of commercial contract review using NLP and machine-learning techniques, while maintaining traceability between analytical findings and the contract text.

- **To ingest supported legal documents safely:** validate PDF and DOCX signatures, handle invalid or corrupt files, and enforce an upload-size limit.
- **To preserve contract structure:** identify section headings, segment documents into clauses, stitch multi-line headings, and reduce orphan fragments created by formatting or page breaks.
- **To identify legal entities and defined parties:** extract contracting party names and roles, effective dates, notice periods, payment terms, governing jurisdictions, and other relevant commercial information.
- **To extract actionable obligations:** represent a contractual commitment through who is responsible, what action is required, when it is due, and where it appears in the source.
- **To classify clause risk:** combine deterministic rules with optional fine-tuned Legal-BERT or zero-shot classification, and assign a risk category and severity.
- **To generate evidence-linked summaries:** prioritize important clauses and produce an executive brief whose statements cite their originating sections.
- **To make processing transparent:** report which engine was used for each stage and clearly mark unavailable model-dependent results instead of fabricating data.

# 3. PROBLEM STATEMENT

Commercial contracts frequently combine lengthy prose with clauses describing liability, indemnification, intellectual property, termination rights, auto-renewal, payment, audit access, and notice requirements. A manual reviewer must locate relevant sections, determine which party has each responsibility, calculate important dates, estimate risk, and communicate the findings to decision-makers. The work can be repetitive and error-prone when many documents must be reviewed.

## Proposed Solutions

- **Hidden or uncapped liability:** identify relevant liability language and classify the corresponding clause under a risk taxonomy.

- **Unclear party responsibilities:** extract contracting parties and normalize actionable commitments into a structured who-what-when record.
- **Missed notice periods and deadlines:** normalize notice durations and calculate term or non-renewal dates using an explicitly stated date convention.
- **Difficult clause navigation:** segment contract sections and preserve source attribution so that findings can be traced to the original clause.
- **Information overload:** select key operative and elevated-risk clauses before generating a concise executive summary.
- **Variable model availability:** apply a deterministic rules layer and explicit degradation modes when advanced machine-learning models are unavailable.
- **Need for transparent review:** attach confidence, rationale, offsets, page/section metadata, and engine telemetry to outputs where available.

# 4. TOOLS AND TECHNOLOGIES USED

| Tool / Technology | Purpose in LexAura |
|---|---|
| **Python 3.11+** | Backend implementation, document processing, NLP pipeline orchestration, date calculations, and model inference. |
| **FastAPI** | REST API for document submission, asynchronous job status polling, health checks, sample analysis, and OpenAPI documentation. |
| **React 19 + Vite** | Web client, dashboard, document analysis views, progress/status presentation, and interactive results. |
| **Hugging Face Transformers** | Transformer pipelines for named-entity recognition, zero-shot classification, and neural summarization when dependencies and model weights are available. |
| **dslim/bert-base-NER** | Fallback transformer pipeline for named-entity recognition and aggregation of subword tokens. |
| **Legal-BERT / CUAD** | Optional fine-tuned legal clause risk classification path; the README describes a training-data conversion pipeline for CUAD. |
| **DistilBART / BART MNLI** | Optional abstractive summarization and zero-shot risk classification models. |
| **PDFPlumber / python-docx** | Text extraction from PDF and DOCX documents as part of document ingestion. |
| **Docker + Docker Compose** | Containerized local deployment for the API and frontend with model-cache binding. |
| **pytest** | Repository tests for API contracts, ingestion validation, entity extraction, normalization, and risk-engine behavior. |
| **CUAD v1** | Contract Understanding Atticus Dataset, used by the documented conversion/training pipeline for clause taxonomy alignment. |

The project also uses asynchronous job tracking, JSON responses, environment-based configuration, regular expressions for structural and term extraction, and Hugging Face model caching. The README describes the job store as TTL in-memory with Redis as an architectural option; database-backed enterprise persistence is listed as a future roadmap stage.

# 5. SYSTEM ARCHITECTURE

LexAura uses a decoupled full-stack architecture. The React client uploads a supported document to the FastAPI server. The server validates and parses it, runs the Legal-NER, risk classification, and summarization stages, stores the asynchronous job status, and returns structured JSON results through progressive polling.

## 5.1 Overall Architecture

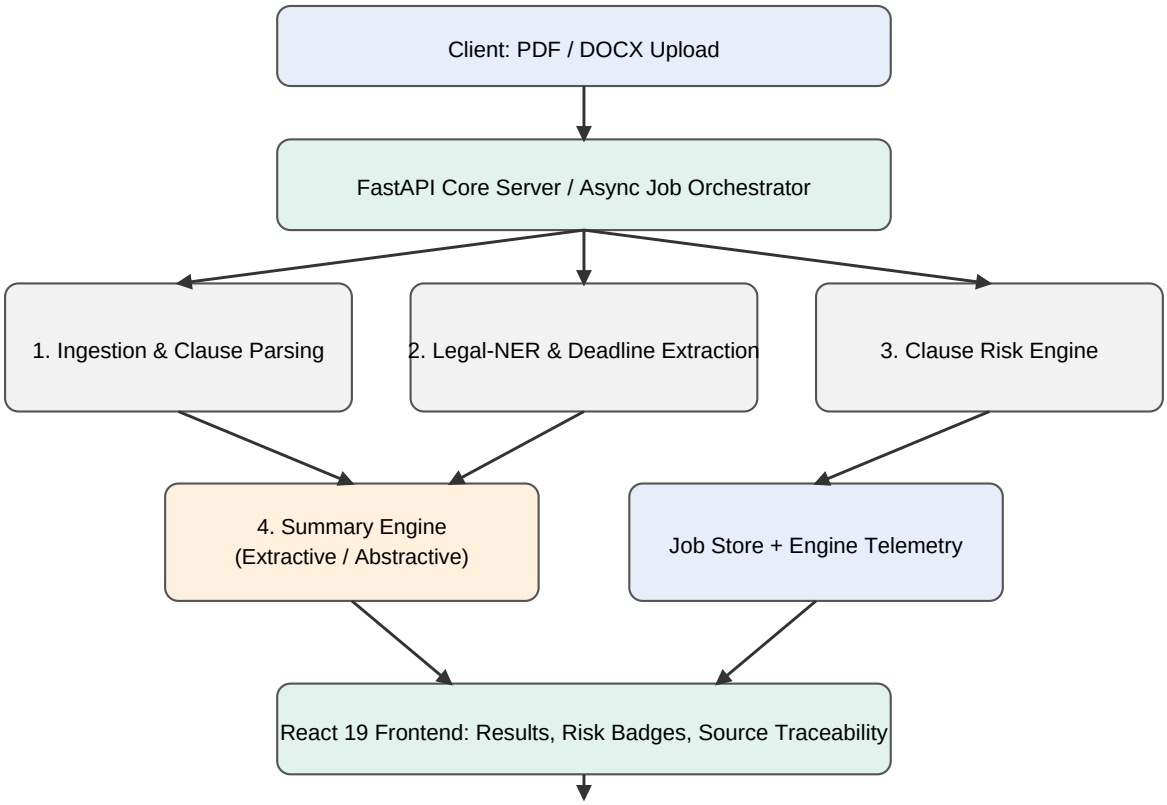



Fig 5.1 Overall architecture of LexAura

## 5.2 User Layer

The user selects a legal agreement in PDF or DOCX format and submits it through the web interface. The frontend then polls the job-status endpoint and presents the analysis results after processing finishes. The README lists a built-in sample Master Services Agreement demonstration endpoint.

## 5.3 Frontend Layer

The frontend is built with React 19 and Vite. The README describes a custom glassmorphism interface, motion transitions, risk severity badges, clause-level source traceability, a document split-pane concept, deadline timelines, and obligation action matrices. Some workspace redesign details are part of the future roadmap rather than necessarily all complete in the current implementation.

## 5.4 Backend Layer

The FastAPI server handles service discovery, health reporting, multipart document upload, asynchronous pipeline dispatch, job polling, and sample analysis. Its main orchestration logic is in *backend/app.py*, with specialized modules for ingestion, NER, risk classification, and summarization.

### 5.5 NLP and Machine Learning Layer

The intelligence layer combines structural regular expressions, defined-term extraction, transformer NER, deterministic risk rules, optional zero-shot or fine-tuned classifiers, and extractive or abstractive summarization. Engine telemetry records which processing path produced an output.

### 5.6 Output and Traceability Layer

The system returns structured results for clauses, severity, category, confidence, rationale, character offsets, page information, detected entities, obligations, deadlines, summary text, and engine status where applicable. Clause-level attribution is used to make the generated summary easier to verify against the original contract.

### 5.7 Data and Job Layer

The repository includes a sample MSA, a CUAD excerpt, test-PDF generation utilities, a profiling script, model-cache support, and an asynchronous job store described as TTL in-memory with Redis as an option. The README marks user-scoped SQL persistence, authentication, and audit logging as a future stage.

# 6. NLP CONCEPTS USED

## 6.1 Document Ingestion and Structural Parsing

Before NLP analysis, LexAura validates the actual file signature rather than relying only on the filename or browser-provided MIME type. PDFs are checked for the %PDF- signature; DOCX files are checked for the ZIP signature and the presence of word/document.xml. A 50 MB upload limit and checks for empty or corrupt payloads are described in the README.

The parser identifies likely numbered clause headings using regular-expression heuristics, joins headings that span multiple lines, filters out common corporate suffixes that could be mistaken for headings, and reattaches or removes short orphan fragments caused by page-break formatting.

**Example:** “ARTICLE II” on one line followed by “TERM AND TERMINATION” on the next line should be treated as one heading rather than two unrelated sections.

## 6.2 Clause Segmentation

Clause segmentation divides a long contract into sections or clauses so that each unit can be analyzed and referenced. Keeping the clause heading and body together is important because the risk engine and summary need context, and the user must be able to navigate back to the relevant provision.

**Example:** A section headed “4.0 UNLIMITED LIABILITY” is retained as an independent clause unit. The title and clause text can be returned together with its assigned category and severity.

## 6.3 Legal Named Entity Recognition (Legal-NER)

Named Entity Recognition identifies meaningful names and values inside text. LexAura combines rule-based defined-term extraction with a transformer fallback. Standard contract preambles such as “between Acme Corp ("Client") and Global Tech ("Service Provider")” can be parsed into entity records with a name and contractual role. When preamble patterns vary, a dslim/bert-base-NER pipeline is described as a fallback, with aggregation to join subword fragments.

**Example:** Acme Corp → Client; Global Tech → Service Provider.

## 6.4 Date Normalization and Notice Period Extraction

Contracts express dates and notice intervals in many textual forms. LexAura normalizes recognized dates into ISO-style YYYY-MM-DD form and converts text durations such as “at least ninety (90) days prior written notice” into numeric durations. Notice types can be categorized as termination, non-renewal, or general notice. The system also looks for payment terms such as “Net 45” and recurring fees or percentages.

**Example:** “November 1, 2026” can be represented as “2026-11-01”; “90 days prior written notice” can be represented as a duration of 90 days.

## 6.5 Obligation Triplet Extraction

An obligation describes an action expected from a party. LexAura aims to represent obligations using the fields who, what, when, and source. This converts a sentence embedded in legal prose into an action-oriented record that can be reviewed by a team.

**Example:** Who: Client; What: pay Service Provider; When: within 30 days of invoice receipt; Source: rule-based extraction.

## 6.6 Pattern Matching and Deterministic Rules

Regular expressions and keyword patterns are used to identify recurring clause structures and high-confidence risk phrases. The rule engine is placed before the heavier model-based classifier and can immediately recognize explicit language such as “unlimited liability”, “termination for convenience”, or auto-renewal patterns. Deterministic rules are useful for predictable phrases, although unusual wording may require a model or human review.

## 6.7 Clause Risk Classification

LexAura documents a multi-tier hierarchy: first, deterministic pattern matching; second, a fine-tuned Legal-BERT classifier when a configured model path is available; third, a zero-shot baseline using a BART/DistilBART MNLI model; and finally an explicit degradation mode if machine-learning dependencies are unavailable. The fallback is intended to disclose the actual engine state rather than create unsupported predictions.

# 6. NLP CONCEPTS USED (CONTINUED)

## 6.8 Risk Taxonomy and Severity

The README defines 13 clause categories. High-severity categories include uncapped liability, intellectual-property ownership assignment, and change of control. Medium-severity categories include termination for convenience, non-compete, exclusivity, and anti-assignment. Low-severity categories include liability caps, renewal terms, notices required to prevent renewal, liquidated damages, and audit rights. Standard or operational provisions may be marked no_risk with severity None.

| Risk category | Meaning | Severity |
|---|---|---|
| uncapped_liability | Unlimited or uncapped liability provisions | **High** |
| ip_ownership_assignment | Assignment or transfer of proprietary IP rights | **High** |
| change_of_control | Rights triggered by merger, acquisition, or sale | **High** |
| termination_for_convenience | Unilateral termination without cause | Medium |
| non_compete / exclusivity / anti_assignment | Competitive restriction, exclusive dealing, or assignment restriction | Medium |
| cap_on_liability / renewal_term / notice_to_terminate_renewal | Liability cap, renewal, or required renewal notice | Low |
| liquidated_damages / audit_rights | Pre-agreed damages or contract audit rights | Low |
| no_risk | Standard boilerplate or operational provisions | None |

## 6.9 TF-IDF and Transformer Models

LexAura's documented core pipeline does not describe TF-IDF as the main clause-risk method. Instead, it uses rule-based patterns, optional transformer classifiers, and transformer summarization. The repository also describes CUAD training-data conversion for clause classification. CUAD labels are mapped into LexAura's normalized categories, while non-annotated paragraphs can be sampled for the no_risk class. The training utility uses contract-level splits (80% training, 10% validation, 10% test) to reduce leakage between partitions.

## 6.10 Summarization and Source Attribution

Long contracts can be expensive to summarize with autoregressive models. LexAura first selects a small set of important clauses, prioritizing high- and medium-risk provisions and operative terms related to liability, termination, payment, or contract term. It can then use a DistilBART summarization model or a deterministic extractive fallback that selects sentences containing obligation-related terms. Each summary sentence is intended to carry a reference to its source section.

**Example:** "Client may terminate this Agreement by providing thirty days' written notice. (Source: 3.0 TERMINATION FOR CONVENIENCE)"

## 6.11 Computed Contract Deadlines

The system calculates dates using stated contractual conventions. The README provides an example in which the term-end date is calculated from the effective date plus one year under a 365-day convention, and the last day for non-renewal notice is calculated by subtracting the normalized notice period. Because contracts may define terms differently, the formula and assumption are included in the output and should be checked against the agreement.

## 6.12 Engine Telemetry and Graceful Degradation

LexAura returns engine telemetry indicating whether a stage used a model, fine-tuned model, zero-shot baseline, extractive fallback, rule engine, or mock/degraded mode. If a model-dependent field cannot be computed, the README says the system should report it as unavailable rather than fabricate a result. Missing entities or risks are explicitly labelled “not found in document”, not implicitly treated as proof that the contract is safe.

# 7. IMPLEMENTATION STEPS

## 7.1 Project Structure and Environment Setup

The project separates the frontend and backend into independent directories. The backend includes the FastAPI app, model download utility, dependency files, core processing modules, and tests. The frontend includes the React application, landing page, styles, and entry point. Dockerfile and docker-compose.yml provide a containerized option; configuration can be supplied through environment variables such as RISK_MODEL_PATH, PRELOAD_MODELS, HF_HOME, and CORS_ORIGINS.

## 7.2 Document Upload and Ingestion

The client submits a PDF or DOCX file to POST /api/analyze using multipart form data. The backend checks size and raw signature, extracts document text, parses clause headings, and structures the document for subsequent steps. A successful asynchronous submission returns a job identifier and a processing status.

## 7.3 Legal Entity and Obligation Processing

The Legal-NER module parses contracting party names and roles, dates, notice periods, payment terms, fees, jurisdictions, and obligation statements. Date and duration normalization converts recognized values to a consistent representation. Deadline calculations include a formula and the term convention used.

## 7.4 Risk Classification

The risk engine begins with deterministic pattern matching for explicit risk phrases. If configured and available, it can use a fine-tuned Legal-BERT classifier or zero-shot model. Results include clause header and text, category, severity, confidence, rationale, offsets, page number, and the engine source where available.

## 7.5 Summary Generation and Result Delivery

The summarizer selects high-value clauses, then uses a neural summarizer or the deterministic extractive fallback. The job endpoint reports progressive status and returns the final result after processing. Every summary item should reference the clause from which the information was taken, making the output auditable.

## 7.6 API Endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| GET | / | Service discovery and endpoint listing. |
| GET | /health | Service health check and per-module engine status. |
| POST | /api/analyze | Submit PDF/DOCX document for asynchronous analysis. |
| GET | /api/jobs/{job_id} | Poll job progress and retrieve result. |
| GET | /api/sample-demo | Run analysis on the built-in sample MSA. |

## 7.7 Testing and Performance Profiling

The repository lists tests for API contracts and health validation, ingestion and file signatures, Legal-NER and normalization, and risk-engine labels and deterministic rules. Performance tools include a script for profiling the API and generated documents of different lengths. The README reports that extractive pre-selection, CPU thread management, and sequential model execution were used to reduce summarization latency and avoid CPU thread contention.

## 7.8 Running the Application

The README recommends running *docker compose up --build -d* or the PowerShell runner *scripts/run_local.ps1*. It lists the frontend at http://localhost:5173, backend API at http://localhost:8000, and interactive OpenAPI documentation at http://localhost:8000/docs. For manual execution, install backend dependencies from requirements.docker.txt and run the backend app, then install frontend packages and run the Vite development server.

# 8. RESULTS

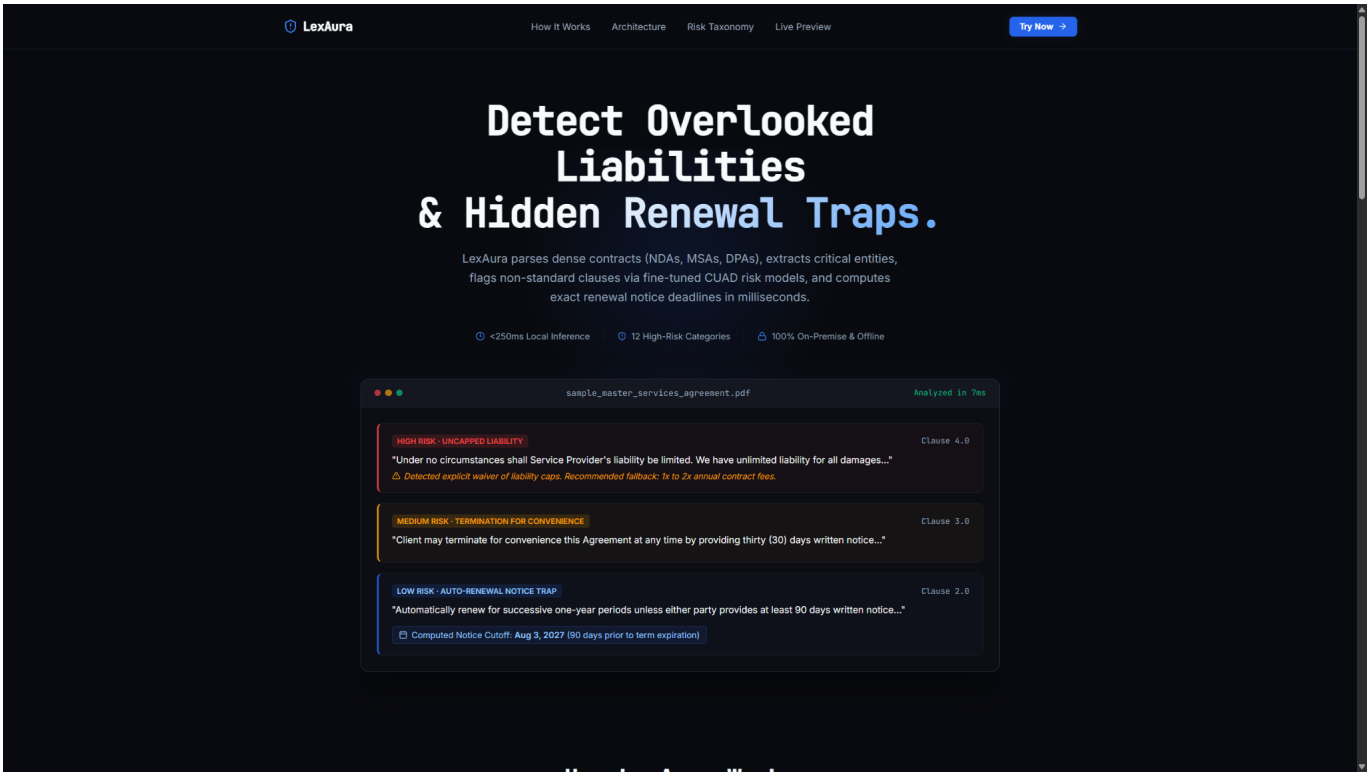



Fig. 8.1 LexAura Landing Page and Project Overview

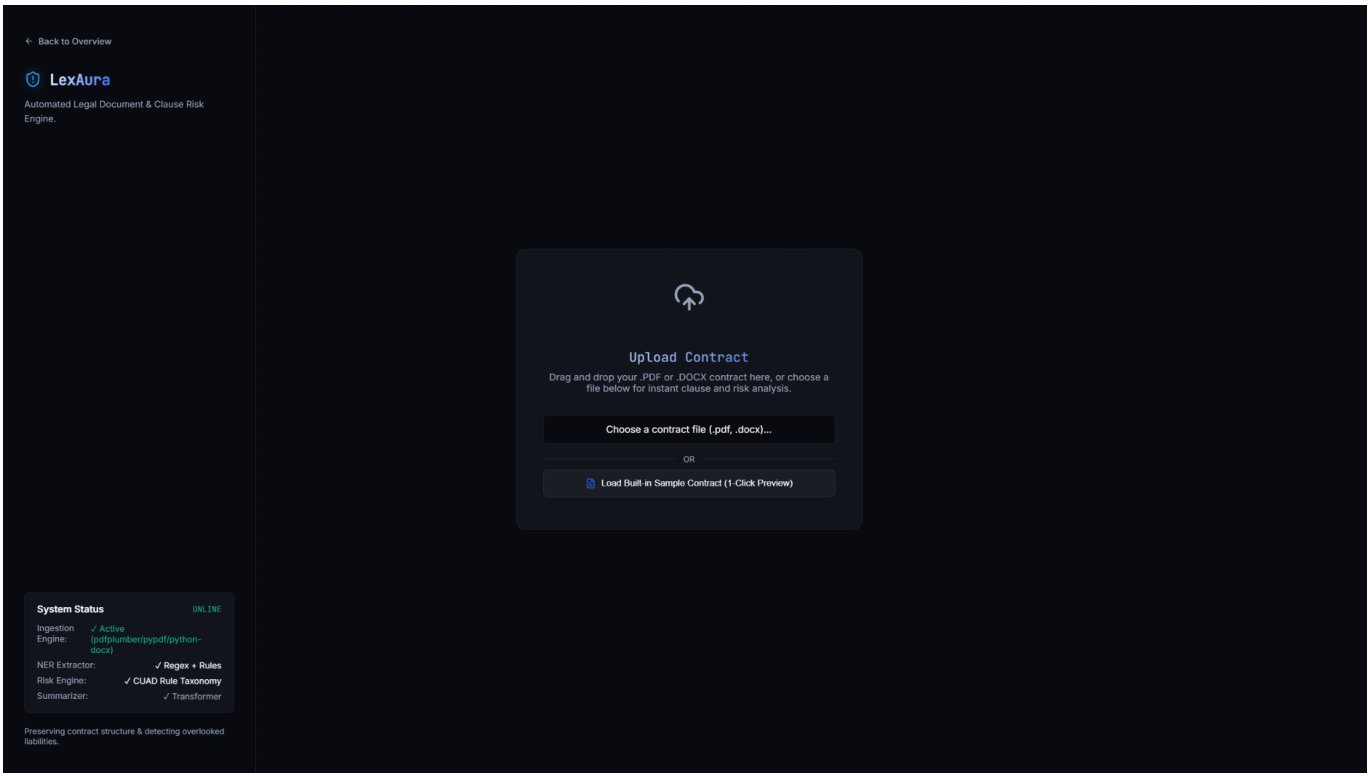



Fig. 8.2 Contract Upload Interface with Built-in Sample Option

# 8. RESULTS (CONTINUED)

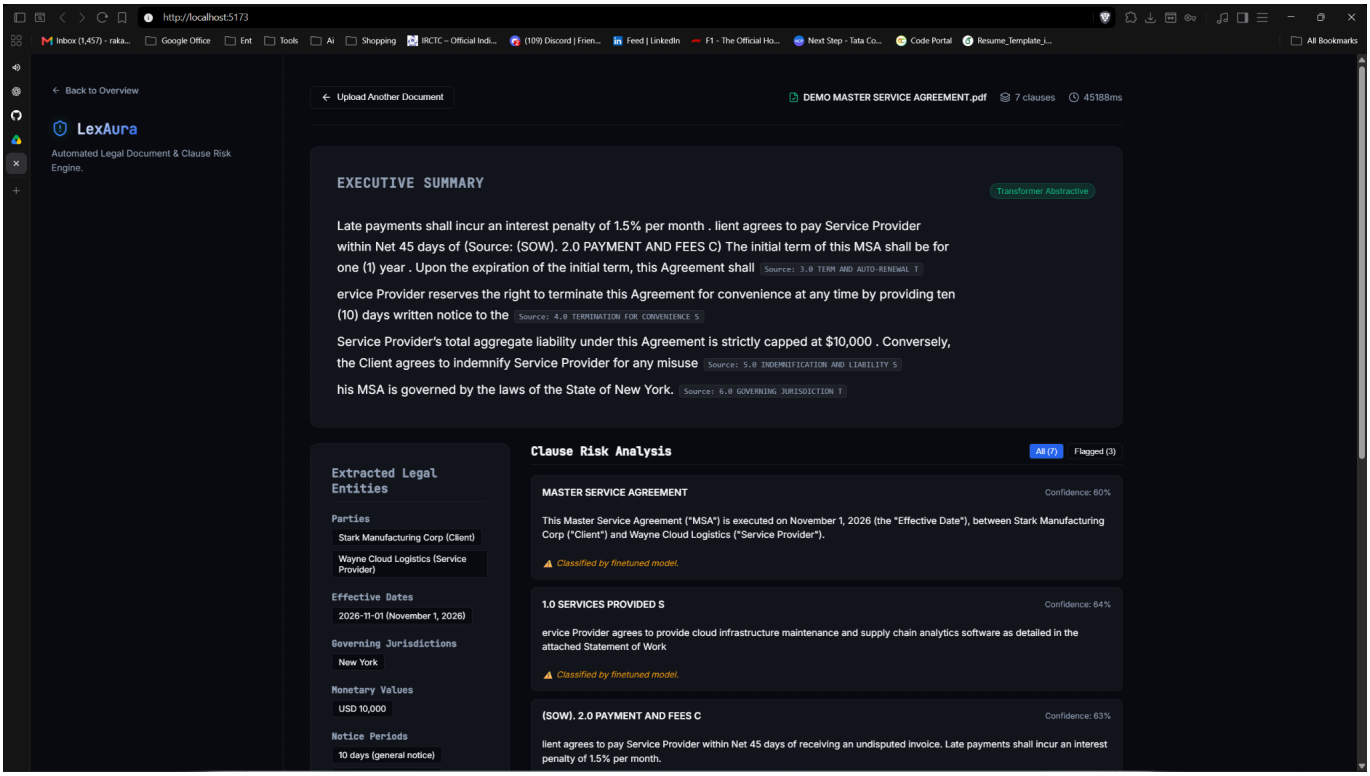



Fig. 8.3 Master Services Agreement: Executive Summary and Extracted Legal Entities

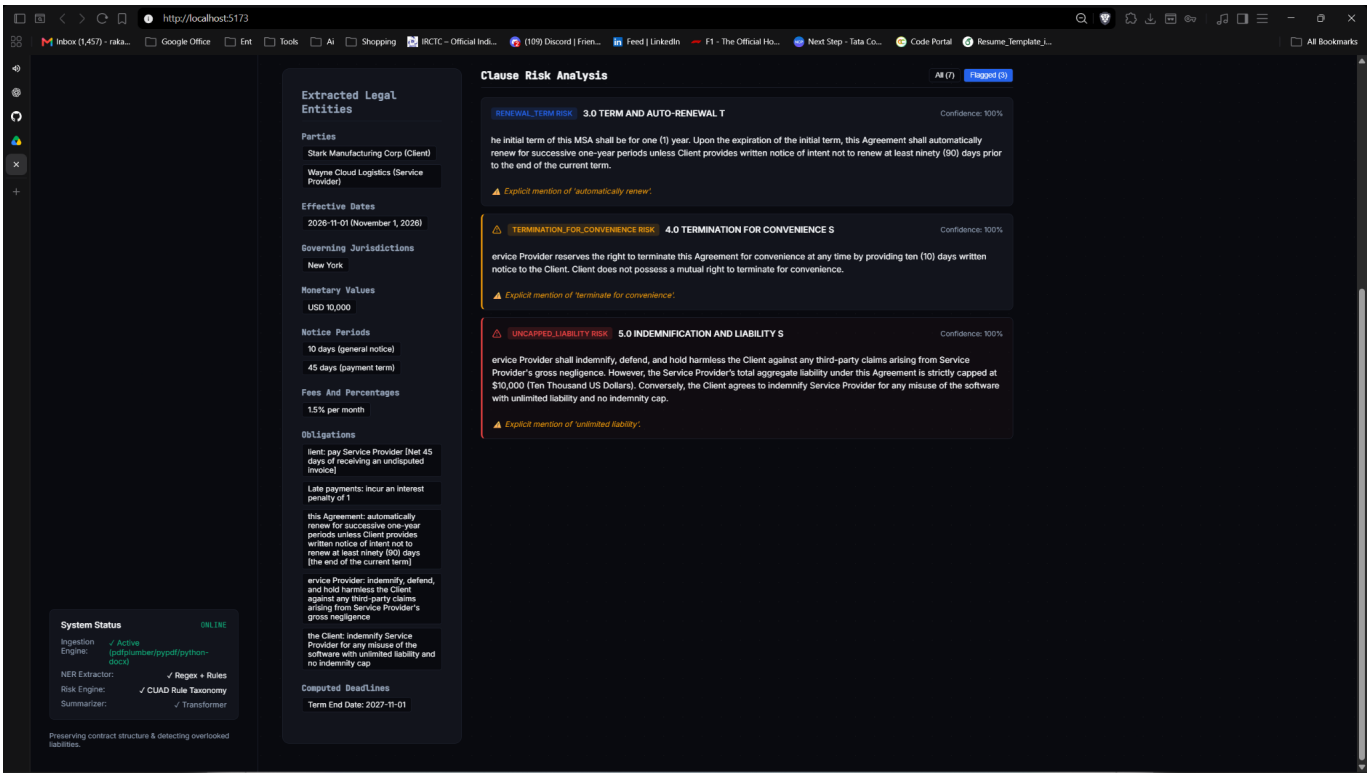



Fig. 8.4 Master Services Agreement: Flagged Clause Risks and Computed Deadlines

# 8. RESULTS (CONTINUED)

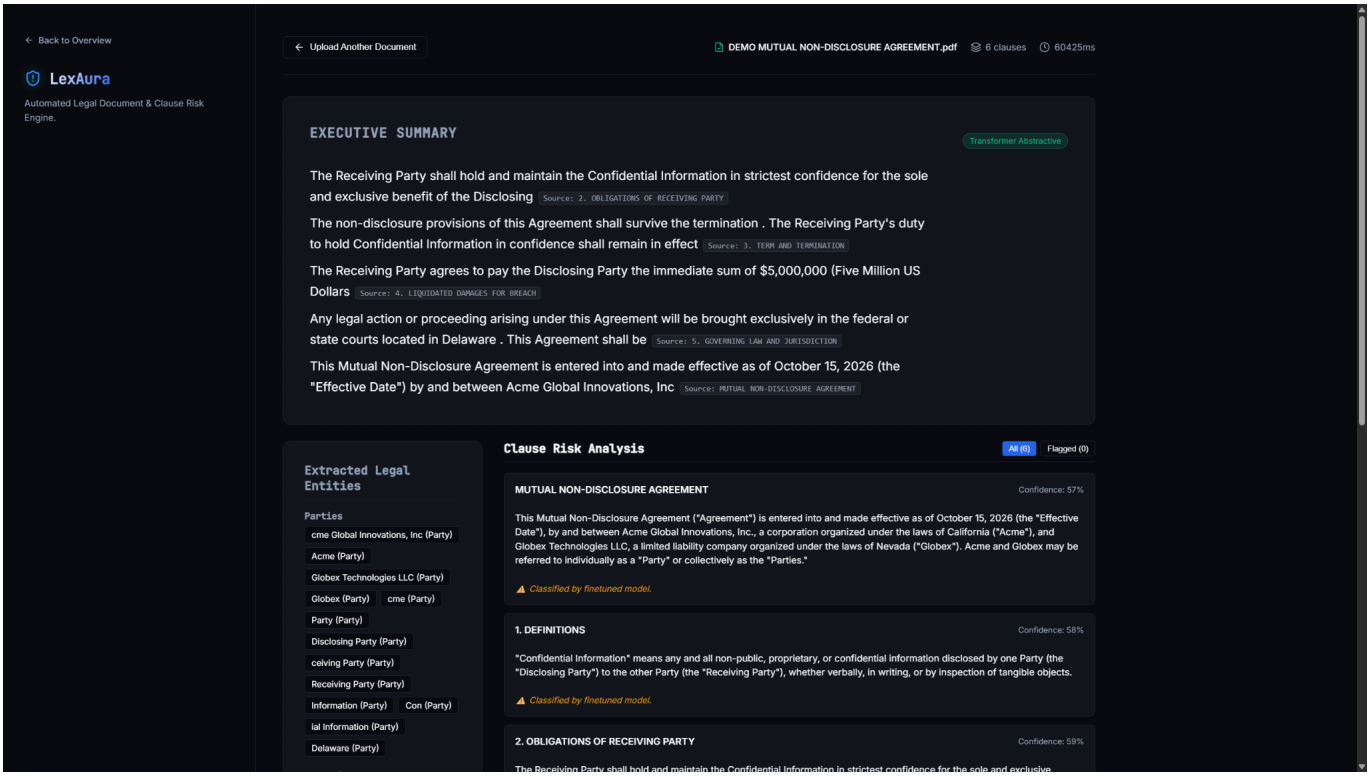



Fig. 8.5 Mutual Non-Disclosure Agreement: Executive Summary and Clause Analysis

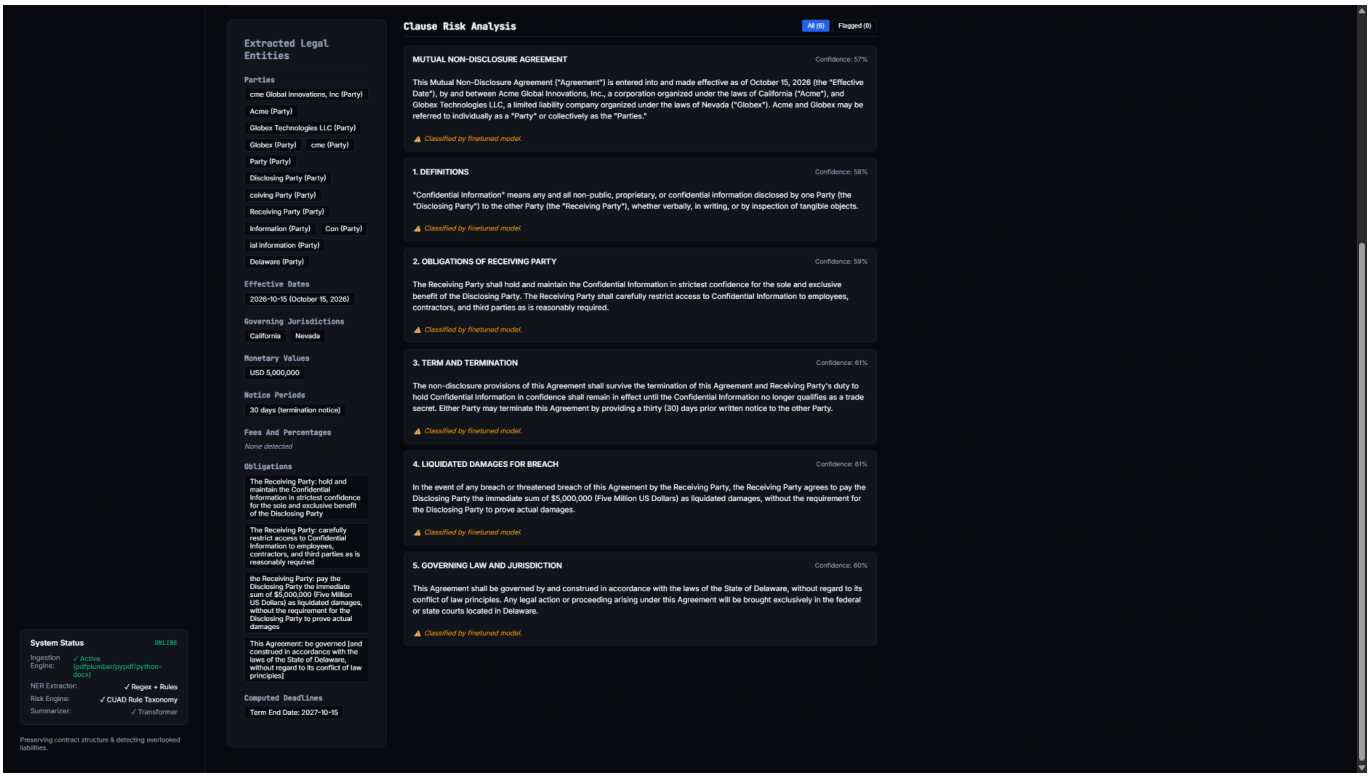



Fig. 8.6 Mutual Non-Disclosure Agreement: Detailed Clause Risk Analysis and Extracted Entities

# 9. FUTURE SCOPE

The repository README presents a staged roadmap. The first two stages—profiling/baseline diagnostics and backend quality/latency optimization—are marked complete. The following items are listed as future work and should not be represented as already completed without separate implementation evidence.

- **Enterprise authentication and persistence:** add SQLite or PostgreSQL storage with user-scoped document ownership, password hashing, secure access and refresh tokens, rate limiting, and audit logs for uploads, viewing, deletion, and exports.
- **Legal-tech workspace redesign:** provide a resizable split-pane PDF viewer and result tabs for summary, risks, entities, obligations, and deadlines; improve accessibility and motion behavior.
- **Contract comparison and semantic diff:** align clauses across contract versions, detect material changes in liability caps, notice windows, payment terms, and other provisions, and classify whether risk increased or decreased.
- **Production hardening and export:** introduce full end-to-end tests, dependency vulnerability audits, executive PDF report export, structured JSON exports, and calendar ICS exports for deadlines.
- **Model and dataset improvements:** continue validating fine-tuned clause classifiers against contract-level splits and improve evaluation, calibration, and category coverage.
- **Auditability and usability:** make engine status, confidence, assumptions, and clause references visible so reviewers can quickly verify generated findings.

# 10. CONCLUSION

LexAura is an NLP-based legal document and clause analysis platform designed to help reviewers make dense commercial contracts easier to inspect. Its documented pipeline combines secure file validation and structural parsing with Legal-NER, date and notice-period normalization, obligation extraction, deterministic and model-based clause risk classification, and extractive or abstractive summarization. A central design principle is traceability: findings should include source clauses, rationale, and engine information wherever available.

The README documents working API routes, separate frontend and backend modules, test files, sample contract assets, and performance profiling utilities. It also describes meaningful limitations and planned work, including model-availability fallback behavior and future authentication, persistence, contract comparison, and export features. Since automated classifications may miss context or interpret legal language incorrectly, LexAura should be treated as an assistance tool rather than a replacement for legal judgment. All outputs should be verified against the original agreement by a qualified reviewer.

# REFERENCES

1. LexAura repository README, “LexAura: Automated Legal Document & Clause Analyzer”, supplied project README and repository documentation.

2. Contract Understanding Atticus Dataset (CUAD v1), as referenced in the LexAura README for legal contract clause taxonomy and training-data conversion.

3. FastAPI documentation: https://fastapi.tiangolo.com/

4. React documentation: https://react.dev/

5. Hugging Face Transformers documentation: https://huggingface.co/docs/transformers/

6. Python documentation: https://docs.python.org/3/

**Project source repository:** https://github.com/RaKa8904/LexAura